Gradient-Based Small-Molecule Hallucination with Protenix-Tiny

2D relaxed graph optimization | Mosaic | Protenix-Tiny | staged molecular feasibility

Current project logic
Stage 0: extend Mosaic, which currently assumes protein/sequence hallucination and does not natively support small-molecule graph optimization, into a small-molecule branch -> Stage 1: use the new branch to stabilize unconstrained optimization of a highly relaxed 2D molecular graph through frozen Protenix-Tiny -> Stage 2: add the minimum explicit chemistry constraints needed for feasible molecules -> Stage 3: add a frozen learned molecular prior (DeFoG-type; exact model/checkpoint to be verified) -> only then expand to richer properties, 3D reference geometry and more ambitious target-specific design.

1. Motivation and project principle

The long-term goal is to perform de novo small-molecule design directly through gradients from an AF3-class model. The project should therefore develop the optimization mechanics in the same model family that will eventually be used for target-specific design, rather than first training a separate affinity classifier and hoping that the optimization behavior transfers.

The central difficulty is that the early state is not a normal molecule. Atom identities and bonds may be highly fractional, connectivity may be uncertain, and there is no meaningful 3D conformer for a graph that is still extremely relaxed. The first implementation should therefore treat the ligand as a differentiable 2D graph and optimize that graph directly through gradients.

A separate engineering step is required before the scientific experiments can begin: Mosaic currently supports protein/sequence hallucination, not small-molecule graph variables. Stage 0 therefore extends Mosaic while reusing its optimization loop, logging and experiment structure as much as possible.

After the framework exists, Stage 1 deliberately studies the unconstrained optimizer before adding chemistry. This stage is expected to require iteration because several moving parts - initialization, parameterization, gradient scale, entropy/fluidity, objective normalization and hard readout - have to work together before it is meaningful to judge molecular-feasibility constraints.

The project is not a model-training project. Keep Protenix-Tiny frozen, avoid retraining unless a concrete blocker appears, and prefer one simple implementation plus controlled matched experiments over broad architecture or hyperparameter searches.

2. Project progression

Stage	What to do	Output
Stage 0 - Mosaic setup	Extend Mosaic from sequence hallucination to fixed-size relaxed small-molecule graphs and build a differentiable Protenix-Tiny interface.	A reproducible small-molecule hallucination branch with gradients to atom/bond variables.
Stage 1 - Unconstrained baseline	Establish the first AF3-native optimization signal and stabilize unconstrained relaxed-graph optimization.	A stable base algorithm, one canonical unconstrained trajectory and a clear map of failure modes.
Stage 2 - Explicit feasibility	Add simple chemistry constraints, starting with valence and adding only what is needed next.	Higher hard-molecule feasibility without destroying topology/identity fluidity.
Stage 3 - Learned prior	Add a frozen 2D molecular-distribution prior (DeFoG-type; exact model/checkpoint selected after compatibility test).	Matched evidence for whether learned molecular regularization improves feasible generation beyond hand-written rules.
Future	Move to 3D/reference geometry, richer desired properties, stronger target-specific design and independent validation.	Extensions only after the 2D constrained optimizer is stable.

3. Stage 0 - extend Mosaic to small-molecule hallucination

Stage 0 is an engineering stage. Mosaic should keep its existing role as the lightweight optimization framework, but it needs a new molecular branch because the current code is built around protein/sequence variables and does not natively represent or optimize small-molecule graphs.

3.1 What has to change in Mosaic

- Add a small-molecule optimization state instead of amino-acid sequence logits. The state should be a fixed-size relaxed graph that remains differentiable throughout the trajectory.
- Keep the existing Mosaic loop where possible: initialization -> forward model -> scalar objective -> backward pass -> optimizer step -> logging/checkpointing. The main change should be what the optimized variables represent and how they are featurized for Protenix-Tiny.
- Support two forward modes from the same branch: ligand only for the first Stage 1 experiments, and protein + ligand for the fallback/target-specific experiments.
- Do not build molecule generation through discrete RDKit edits inside the optimization loop. The optimization state is continuous; hard molecules are readouts/evaluation states until later stages.

3.2 V0 relaxed molecular representation

Component	V0 representation	Implementation rule
Atom identity X	Per-atom probability distribution over a small element vocabulary.	Optimize probabilities directly with projected-simplex updates; use a fixed number N of atom slots initially.
Bond state E	Per-pair categorical distribution: NO_BOND / SINGLE / DOUBLE / TRIPLE / AROMATIC.	Keep E symmetric and force diagonal pairs to NO_BOND. Protenix-Tiny initially receives bond existence B_ij = 1 - P(NO_BOND); retain full E for later valence/prior terms.
Formal charge q	Fixed at 0 in V0.	Do not optimize charge until X/E optimization is stable.
Ligand ref_pos	All-zero placeholder.	Mask/disable ligand reference-geometry-derived contributions so zeros mean "no ligand 3D information", not "all atoms occupy the same point".
Atom count	Fixed N.	Do not optimize atom existence or molecule size in the first implementation.

3.3 Protenix-Tiny wrapper

- Load Protenix-Tiny once and keep it resident/warm during iterative optimization. Freeze all model parameters; gradients should flow only to the relaxed ligand variables.
- Start with the cheapest useful AF3-class path: the native input/trunk/pair representation and distogram/contact outputs. Do not run coordinate diffusion or the confidence head unless a chosen objective genuinely requires them.
- Keep protein features completely native when a protein is present. Only the relaxed ligand uses the 2D geometry-off path.
- Preserve autograd in evaluation mode. Do not use inference/no-grad wrappers that disconnect X/E from the objective.
- If reducing recycle count is technically supported and stable, use the cheapest setting for Stage 0/1; do not spend time benchmarking many model-depth/recycle variants.

3.4 Stage 0 validation

- Run one ordinary hard molecule through the molecular branch as a sanity check, then run a relaxed version of the same graph.
- Confirm forward + backward completes without NaN/Inf and that atom variables receive non-zero gradients.
- Perturb a bond probability and confirm the objective changes through the Protenix bond-existence pathway.
- Check a few representative derivatives against finite differences to catch silent detach/masking errors.
- Save raw X/E probabilities, optimizer state, RNG state, objective values, gradient norms and periodic hard readouts at every experiment.

Stage 0 stopping rule: once a reproducible ligand-only and protein+ligand forward/backward path exists, stop improving the framework and move immediately to Stage 1.

4. Stage 1 - build the unconstrained baseline optimizer

Stage 1 is the first scientific stage. Do not add valence, connectivity, sparsity or learned molecular-prior terms yet. The purpose is to make one unconstrained relaxed-graph optimizer behave predictably and to identify exactly what breaks before chemistry constraints are introduced.

Work sequentially. Each item below is a gate for the next one: first establish a usable AF3-native signal, then define one canonical relaxed state and initialization, then verify gradients, then stabilize the update dynamics, and only then inspect hard molecular readouts. Do not turn this stage into a broad multi-start or hyperparameter search.

4.1 First signal - use native intra-ligand PAE if it survives the relaxed setup

The first attempt should be a true AF3-class confidence metric, not a new proxy invented for this project. AlphaFold 3 defines PAE for small molecules: each ligand atom is a token and the model predicts the aligned error between ligand token pairs using ligand atom frames constructed from a reference conformer.

Use the exact scalar objective:

L_ligand_PAE = mean_{i != j}(PAE_ij), calculated over all off-diagonal ligand-ligand token pairs with valid frames. PAE is asymmetric, so include both directions i -> j and j -> i. Lower is better; minimize L_ligand_PAE.

Do not use ligand pLDDT for this purpose. In AF3, ligand pLDDT measures ligand-to-polymer local accuracy and therefore is not an intrinsic ligand-only confidence metric.

4.1.1 One quick feasibility gate for ligand-only PAE

First run one ordinary hard ligand through native Protenix-Tiny with its normal generated reference conformer and confirm that token_pair_pae is finite. This verifies the ordinary small-molecule PAE path.

Then run the same molecule through the Stage 0 relaxed branch: ligand ref_pos = 0, ligand geometry features disabled, and X/E exposed as differentiable variables. Check three things only: (1) ligand-ligand PAE remains finite, (2) the relevant ligand tokens still have valid frames, and (3) the mean ligand PAE has non-zero, numerically stable gradients to both X and E.

This is intentionally a single implementation check. AF3 ligand PAE depends on ligand frames derived from the reference conformer, and standard AF3 documentation notes that when ligand reference coordinates cannot be generated and are all zero, ligand confidence values become invalid/NaN. Therefore it may not be compatible with the deliberately geometry-free Stage 0 representation.

If any of the three checks fail, do not derive distogram entropy, do not design another ligand-only confidence score, and do not spend time modifying the confidence head. Switch immediately to the target-based objective below and use that as the Stage 1 baseline.

4.1.2 Default fallback - hallucinate against BRD4-BD1

Use the first bromodomain of human BRD4 (BRD4-BD1) as the debugging target. The 3MXF structure can be used as the target structure, with JQ1 removed. Do not define a binding pocket initially: every BRD4-BD1 residue/token is an eligible contact partner. The goal at this stage is only to test whether the relaxed ligand graph can be optimized toward interaction with a protein target.

Do not add pocket preprocessing in Stage 1. Use the complete BRD4-BD1 target so the first target-based experiment has the minimum possible setup.

Use the Protenix native contact probabilities derived from the distogram. Let c_ij be the probability that ligand atom/token i contacts BRD4-BD1 residue/token j using the native <8 A contact definition.

Optimize one simple ligand-to-target contact score:

S_contact = mean_i max_{j in target}(c_ij). Maximize S_contact.

For each ligand atom, take its strongest predicted contact with any target residue and average across ligand atoms. This is deliberately only a debugging objective: it does not prescribe an epitope or require coverage of the target surface. No affinity classifier, pocket term or additional learned head is used.

Once the protocol chooses ligand PAE or BRD4 whole-target contact optimization, freeze that Stage 1 objective. Do not alternate between objectives while tuning the optimizer.

4.2 Canonical relaxed state and initialization

Use one canonical highly relaxed start first. The goal is to make the optimizer form structure from a fluid graph, not to hide the problem inside a preselected valid topology.

- Fix N (proposed V0 default: N = 25 heavy-atom slots). Keep all atom slots present; do not optimize molecule size in Stage 1.
- Atom state: X_i is a probability simplex over the restricted element vocabulary. Initialize from a broad carbon-heavy probability prior plus small independent probability-space perturbations, then project back onto the simplex so atoms are relaxed but not perfectly permutation-symmetric.
- Bond state: E_ij is a probability simplex over NO_BOND / SINGLE / DOUBLE / TRIPLE / AROMATIC. Protenix receives B_ij = 1 - E_ij,NO_BOND; the full E tensor is retained for later chemistry constraints.

- Initialize the expected bond-existence probability close to p0 = d0/(N-1), with d0 approximately 2.0-2.5. For N = 25 this is roughly 0.08-0.10 per pair. This gives a highly diffuse graph while keeping the total initial bond mass molecule-sized rather than starting from a nearly complete graph.
- Initialize and perturb only one bond-probability vector for each unordered atom pair i < j (the upper triangle). Add small random perturbations there, project each vector back onto the bond-type simplex, then mirror the upper triangle to construct E_ji = E_ij. Keep every diagonal entry fixed to NO_BOND. Atom probabilities receive their own small independent perturbations followed by simplex projection.
- Keep formal charge fixed at zero and ligand ref_pos at the Stage 0 geometry-off zero placeholder. Do not add more optimization variables yet.

Diagnostic fallback only: if the canonical diffuse graph cannot move at all despite healthy gradients, run one soft-chain or soft-tree control to determine whether the specific difficulty is topology formation. Do not make multiple initialization families the main Stage 1 experiment.

4.3 Key considerations for the first optimizer

Build the first optimizer in the order below. The first block contains the pieces that must be implemented in V0; the second fixes the initial values/settings; the third contains behaviors to monitor from the first runs and constraints to add only if the unconstrained baseline shows they are needed.

A. Implement first

- 1. Probability-space optimization.
  - Optimize atom probabilities X and bond-type probabilities E directly; do not use logits + softmax in the first implementation.
  - After every gradient step, project each atom vector and each bond vector back onto its probability simplex.
  - Use the same projected-probability logic already used in Mosaic so probability mass can move directly between atom identities and bond states.
- 2. Optimizer choice.
  - Start with projected gradient updates directly on probabilities, following Mosaic.
  - Do not benchmark multiple optimizers initially; change the optimizer only if projected probability-space updates show a concrete failure.
- 3. Objective.
  - Use one simple AF3-native objective while developing the optimizer.
  - First test mean intra-ligand PAE. If it is not finite/useful with the relaxed ref_pos = 0 branch, switch immediately to the predefined BRD4-BD1 whole-target contact objective.
  - Do not introduce additional proxy objectives until the base optimizer is stable.
- 4. Bond symmetry / representation.
  - Use only the upper triangle of E as learnable bond variables: one probability vector for each unordered atom pair i < j.
  - After every projected-gradient update, project these vectors onto the bond simplex and mirror them to construct E_ji = E_ij; do not optimize E_ij and E_ji independently.
  - Keep all diagonal entries fixed to NO_BOND throughout. This makes bond symmetry and self-bond exclusion representation invariants rather than penalties.
- 5. ref_pos.
  - Initialize ligand ref_pos = 0 and keep ligand reference-geometry contributions disabled during the initial 2D relaxed optimization.
  - The early graph is intentionally too diffuse to have a meaningful conformer. Reintroduce conformer/reference-position information only once the graph is sufficiently molecule-like.

B. Set the baseline values

- 6. Atom vs bond update scale.
  - Treat X and E as separate variable classes and monitor their gradient magnitudes independently.
  - Allow separate step sizes eta_X and eta_E so atom identities and topology both evolve; neither pathway should remain effectively frozen.
- 7. Charge.
  - Keep formal charge fixed at 0 in V0.
  - Only introduce charge optimization after atom identity and topology optimization are stable, since charge adds another easy continuous exploitation direction.
- 8. Atom vocabulary.

  - Use the small initial vocabulary {C, N, O, S, F, Cl}.
  - Expand it only after the first optimizer works and there is a clear chemical reason to add additional elements.
- 9. Fixed molecule size.
  - Keep the number of atom slots fixed within a trajectory; do not optimize atom existence in V0.
  - Start with one simple size (for example N = 25) and later compare a small set such as N = 15, 20, 25, 30.

C. Monitor from the first runs; intervene only if needed

- 10. Fluidity - inspect from the first run.
  - Atom and bond probabilities should remain movable rather than becoming effectively one-hot after a few steps.
  - If premature collapse occurs, first reduce update size / projection aggressiveness; only then consider entropy or other fluidity control.
- 11. Escaping early topology.
  - Track whether strong early edges can later decrease and whether initially weak edges can become dominant.
  - The optimizer must be able to change its mind. If topology becomes irreversible early, fix this before interpreting the objective.
- 12. Valence - later, but monitor from the beginning.
  - Do not constrain valence in the first unconstrained baseline.
  - Calculate expected/argmax valence violations during the trajectory so the dominant chemistry failure is visible.
  - If over-valence is a major failure mode, valence becomes the first explicit chemistry constraint in Stage 2.
- 13. Connectivity - later, but monitor fragmentation.
  - Do not force one connected component in the first baseline.
  - Inspect periodic hard readouts for fragmentation. If disconnected components appear systematically, add a topology-neutral connectedness constraint in Stage 2.
- 14. Sparsity / graph density - later if needed.
  - Track total bond mass and expected degree from the first trajectory.
  - Do not add an explicit sparsity loss unless graphs become excessively dense and valence constraints do not already control the problem.
- 15. Hardening / soft-to-hard gap.
  - Do not harden or repair the graph inside the early optimization loop.
  - Periodically inspect argmax(X) and argmax(E) only as diagnostics, including the soft-to-hard objective gap.
  - Explicit hardening/projection is added later after the relaxed baseline is understood.

4.4 Stage 1 acceptance gate

- The chosen AF3-native objective can be differentiated reliably to both atom and bond variables.
- A short and then extended unconstrained trajectory runs without NaN/Inf, gradient explosions or immediate probability saturation.
- The objective improves reproducibly within a single canonical trajectory and the graph state changes in an interpretable way.
- A raw hard readout is defined and the soft-to-hard score gap plus feasibility failures can be measured.
- Once these conditions hold, stop tuning Stage 1. Chemical feasibility is intentionally unresolved and becomes Stage 2.

5. Stage 2 - add explicit molecular-feasibility constraints

Stage 2 starts from the frozen Stage 1 optimizer. Keep the AF3 objective and probability-space optimizer unchanged; add only the minimum chemistry operators needed to stop obvious failure modes while preserving topology fluidity.

The default Stage 2 loop should stay simple: AF3 gradient step -> simplex projection -> valence projection. Connectivity and sparsity are added only if the corresponding failure remains visible.

5.1 Step 0 - keep hard representation rules as invariants

These remain construction rules, not losses: optimize one upper-triangle bond-probability vector per unordered pair; mirror to E_ji = E_ij; keep diagonal entries fixed to NO_BOND; keep X and E on their probability simplexes; keep N and the atom vocabulary fixed; keep formal charge at zero initially.

5.2 Step 1 - apply valence projection after every update

Use the full soft bond-order tensor E. Assign bond-order values o = {0, 1, 2, 3, 1.5} to NO_BOND / SINGLE / DOUBLE / TRIPLE / AROMATIC and calculate expected soft valence v_i = sum_{j != i} sum_b o_b E_ij,b.

Calculate the allowed valence ceiling from the current soft atom identity: v_max,i = sum_a X_i,a V_a. Start with a deliberately simple neutral table, for example C=4, N=3, O=2, S=2 and F/Cl=1.

After the normal gradient step and simplex projection, project E to the closest bond-probability state satisfying v_i <= v_max,i for all atoms. Preserve the upper-triangle parameterization and each edge simplex. Do not add a separate valence loss in the first implementation.

The projection should remove only excess valence, not force atoms to fill their maximum valence. Monitor whether it becomes too aggressive and freezes topology changes.

5.3 Step 2 - add connectivity only if fragmentation remains a real failure

First inspect the raw argmax graphs after valence projection. If they are repeatedly split into multiple components, add the simplest connectivity correction; otherwise do nothing.

A simple correction is to identify the connected components of the current hard graph and use the existing soft bond probabilities to choose the minimum number of inter-component edges needed to connect them. Increase those selected edges toward a bonded state, then reapply simplex and valence projection.

The purpose is only to remove persistent fragmentation. Do not prescribe a chain, tree, ring system or other target topology.

5.4 Step 3 - add sparsity only if diffuse edge mass remains a real failure

Valence projection already limits local bond order, so do not add a sparsity loss automatically. Monitor whether bond probability remains spread across many weak edges even when valence is satisfied.

If this happens, add one simple weak-edge shrinkage/proximal step after valence projection: move a small amount of low-confidence bonded probability back toward NO_BOND, then reproject the edge simplex and valence constraints.

Use one interpretable sparsification mechanism only. Do not stack edge-count, degree, L1 and bond-mass penalties.

5.5 Step 4 - test hardening as a separate Mosaic-style experiment

Do not mix hardening into the baseline Stage 2 optimizer. Start from the same relaxed checkpoint and compare a normal relaxed continuation against a hardening continuation.

Reuse the existing Mosaic hardening / one-hot transition logic rather than inventing a new rule. Keep the underlying probabilities as the optimized state while making the forward representation progressively more discrete or one-hot.

Measure the immediate objective drop at hardening, whether continued optimization recovers the score, raw molecule validity, and whether the topology becomes irreversibly locked.

Keep simple argmax hardening as the evaluation readout throughout, but treat the Mosaic-style hardening continuation itself as a separate experiment.

5.6 Stage 2 experiment order and stopping rule

Experiment A: frozen Stage 1 baseline + valence projection.

Experiment B only if needed: add connectivity correction when fragmentation remains systematic.

Experiment C only if needed: add weak-edge sparsification when diffuse bond mass remains despite acceptable valence.

Experiment D: separate Mosaic-style hardening test from a matched relaxed checkpoint.

Stop adding hand-written chemistry rules once the graph is substantially more molecule-like without losing useful atom/topology movement. Stage 3 should provide richer molecular-distribution pressure rather than accumulating many manual constraints.

6. Stage 3 - add a learned molecular-distribution prior

Stage 3 asks whether a pretrained 2D molecular graph generative model can continuously regularize the relaxed graph toward molecular space at every optimization iteration. The prior stays frozen; this should not become a new model-training project.

6.1 The exact molecular prior is intentionally not selected yet

- Candidate examples include DeFoG, DiGress, or another pretrained 2D atom-and-bond denoiser / flow model.
- Select the model based on practical compatibility: atom/bond vocabulary, fixed-size/padding support, pretrained drug-like checkpoint, speed, and ability to return atom- and bond-level denoising predictions for noisy graphs.
- Do not retrain a new molecular prior unless no suitable pretrained model can be made compatible.

6.2 First integration - use the prior as a moving teacher

At every optimization iteration, take the current relaxed graph (X,E) and query the frozen molecular model once. The model returns cleaner atom and bond distributions (X_hat,E_hat).

For a diffusion/denoising model, this means using one denoising prediction at a chosen noise level; do not run a complete diffusion sampling trajectory at every AF3 optimization step.

Treat (X_hat,E_hat) as a stop-gradient teacher and add a simple discrepancy term that pulls the current X and E toward the predicted cleaner distributions. Recompute the teacher from the new graph at the next iteration.

The optimization loop therefore remains: AF3 objective + molecular-prior penalty -> gradient step -> simplex projection -> valence projection. Connectivity/sparsity operators remain conditional as in Stage 2.

Start with one moderate prior weight. The prior should improve molecular correctness without immediately overwriting the topology changes requested by the AF3 objective.

6.3 Stage 3 success criteria

- Higher raw hard-molecule validity and fewer severe valence/fragmentation failures than Stage 2 alone.
- Smaller soft-to-hard score gap and fewer obviously adversarial fractional states.
- No major collapse of the AF3 objective and no premature freezing of the relaxed graph.
- If these conditions are not met, first revisit prior strength/model compatibility rather than adding a more complicated diffusion schedule.

7. Future extensions - only after Stages 0-3 are stable

- Move from ligand-only optimization to target-specific protein-ligand design if Stage 1 remained ligand-only; first use well-characterized targets, then novel targets.
- Reintroduce 3D/reference geometry only after the graph becomes sufficiently discrete: harden/project the graph, generate a conformer, enable the ligand geometry pathway, and quantify the 2D -> native 3D score gap.
- Add desired-property objectives. Start with features well-defined on relaxed graphs (molecular weight, composition, net charge); later add bond-order-dependent or learned properties such as HBD/HBA, TPSA, logP, solubility, permeability or ADMET. Prefer target ranges / flat-bottom penalties over unconstrained maximization.
- Optimize formal charge, variable molecule size, larger element vocabularies and richer bond chemistry only after the fixed-size neutral baseline is stable.
- Explore property-conditioned molecular priors instead of accumulating many independent property losses.
- Scale from Protenix-Tiny to larger Protenix/Boltz/AF3-class models and independently evaluate hardened molecules with a model not used for optimization.
- Extend to multi-objective design: target interaction, specificity, drug-like property profile, synthetic feasibility and application-specific constraints.

8. Minimal project outputs

- Stage 0: a Mosaic small-molecule branch that optimizes relaxed atom/bond variables through frozen Protenix-Tiny, with ligand ref_pos = 0 and ligand reference-geometry-derived features disabled during the 2D phase.
- Stage 1: a reproducible unconstrained baseline using native mean intra-ligand PAE only if it is finite/differentiable in the relaxed geometry-off wrapper; otherwise BRD4-BD1 whole-target contact hallucination is the default. Include the complete sequential optimizer-development diagnostics, one canonical relaxed initialization and documented failure modes.
- Stage 2: a matched feasibility baseline using valence projection first, with connectivity/sparsity added only if needed, plus a separate Mosaic-style hardening experiment.
- Stage 3: a matched moving-teacher molecular-prior experiment using a still-to-be-selected pretrained 2D denoiser/flow model (for example DeFoG or DiGress).
- A short final report identifying which mechanisms improve feasible hard molecules without eliminating useful gradient-based exploration.

9. Starting references / implementations

Protenix repository and model implementation: https://github.com/bytedance/Protenix

DeFoG: Discrete Flow Matching for Graph Generation - official implementation: https://github.com/manuelmlmadeira/DeFoG

DiGress: Discrete Denoising diffusion for graph generation - https://arxiv.org/abs/2209.14734

DBMol: structure-predictor-guided small-molecule design with optimization + molecular projection: https://arxiv.org/abs/2607.19237

AlphaFold 3 confidence/output documentation (including small-molecule PAE): https://github.com/google-deepmind/alphafold3/blob/main/docs/output.md

AlphaFold 3 ligand conformer/reference-coordinate behavior: https://github.com/google-deepmind/alphafold3/blob/main/docs/input.md

BRD4-BD1 target structure: RCSB PDB 3MXF - https://www.rcsb.org/structure/3MXF